

Since Student A is the “**Integration Specialist**,” your job is to build the foundation. You are responsible for making sure the computer can “hear” and “understand” sound.

Follow these steps exactly. Don’t worry about the code logic yet—focus on the **setup** and **capture**.

Step 1: The Project “Backbone” (Setup)

Before you write a single line of code, you need to give Java its “Ears.”

1. **Download the Model:** Go to the [Vosk Models page](#) and download `vosk-model-small-en-us-0.15.zip`.
 2. **Unzip:** Create a folder in your Java project named `model` and extract the files there.
 3. **Add Libraries:** Download the Vosk and JNA .jar files. In IntelliJ, go to **File > Project Structure > Libraries** and add them. This allows Java to use the speech recognition tools.
-

Step 2: The `VoiceListener` Class (The Specialist)

Create a new Java class named `VoiceListener.java`. Your goal here is to handle the hardware.

1. **Define the Format:** You must tell the microphone to record in a way the model understands.
 - o *Task:* Set the Sample Rate to **16000.0f**, Sample Size to **16 bits**, and Channels to **1 (Mono)**.
 2. **Open the Line:** Use Java’s `AudioSystem` to find your laptop’s microphone (called a `TargetDataLine`).
 3. **Start the Mic:** Write a method called `startMic()` that opens the line and begins “pulling” audio from the air.
-

Step 3: The “Translation” Logic

Inside the same class, you need to connect the microphone to the Vosk “Brain.”

1. **Initialize the Recognizer:** Create a Vosk Recognizer object using the model folder you downloaded in Step 1.
 2. **The Reading Loop:** Create a loop that reads “chunks” of audio (bytes) from the microphone and feeds them into the Recognizer.
 3. **Get the String:** The Recognizer will return a JSON string. Your job is to extract the plain text (e.g., “open notepad”) from that JSON.
-

Step 4: The “Hand-off” (Final Task)

To finish your half of the work, you need to create a simple way to give the text to Student B.

1. **Create a Method:** public String listenForCommand().
 2. **The Result:** When Student B calls this method from the Main class, it should wait until someone speaks, stop the mic, and return the recognized words as a simple Java String.
-

Summary of Student A’s Deliverables:

- ☐ A project folder containing the model and the necessary .jar libraries.
 - ☐ A working VoiceListener class that can turn on the mic.
 - ☐ The ability to print “You said: [Something]” in the console.
-

How to know you are done:

If you can run your code, speak into your laptop, and see your words printed in the IntelliJ/Eclipse console, **your work is 100% complete**. You can then hand that “String” to Student B.

Student A, does this path make sense? Once you feel confident, we can go over the steps for Student B!